

Phase 1 — Domain & Geometry Implementation

Contract

Status: Implementation contract (Rev 3)

Scope: Pure TypeScript spatial engine only. No React, Konva, R3F, Zustand UI, or renderer code.

Goal: Produce a fully testable, renderer-independent domain + geometry core that answers:

Can this furniture item be placed at this position, size, rotation and elevation in this room, given the current furniture, walls and fixtures?

Rev 3 changes: stable vertex IDs replace array-index wall topology (§3, §9.18), composite atomicity (§5a), tolerance-based quantization assertion (§6), already-invalid-object escape rule (§6), mandatory arc discretization constant (§7), domain-owned `resizeAboutAnchor` helper, optional AABB prefilter permitted in Phase 1, multi-room shared-wall topology explicitly noted as a future-boundary concern.

Rev 2 changes: `apply` failure semantics + error codes (§5a), mandatory inverse data + inverse-application undo (§4, §5b), EditWall pipeline exception (§6a), fixture vertical validity, door-vs-door soft violation, clearance zone clipping, rotation storage note, history interface (§5c).

1. Non-negotiable Invariants (from V2)

- Domain owns all authoritative state.
 - No screen coordinates, viewport, or DPR in domain.
 - Linear values quantize to 1 mm; rotation quantizes to nearest 0.1° (or active angular snap).
 - Rounding mode: `Math.round` semantics on the millimetre / 0.1° value. Midpoint 0.0005 m → 0.001 m; negatives follow the same magnitude rule.
 - `EPSILON = 1e-6 m` for geometric predicates.
 - Room polygon = interior finished face (CCW), simple polygons only (no holes).
 - Hard constraints block commit; soft constraints do not.
 - One continuous operation → one immutable command. Multi-object operations are a single `CompositeCommand`.
 - `apply(command, state)` is a pure function. All new IDs are generated *outside* `apply` and carried inside the command.
 - Commands contain no renderer or transient interaction state.
 - Commands arriving at `apply` are **already quantized**; `apply` asserts but never repairs.
 - `apply` is **total over well-formed commands**: constraint rejection happens in the pipeline before a command is emitted; `apply` throws only on malformed commands (stable error codes, §5a).
 - Undo = **inverse-application**; every command carries mandatory inverse data (§4).
 - Boundary policy: entirely inside or exactly touching = valid (unless clearance forbids it).
 - Validity is *derived* by re-validation after every committed change; no persistent `isInvalid` flag.
 - Locked objects still participate in collision/containment; move/resize/rotate/delete on them are rejected in the pipeline.
-

2. Core Types

```
type Metres = number; // canonical internal unit
interface Vec2 { x: Metres; y: Metres }
interface Vertex { id: string; position: Vec2 } // stable ID; never referenced by
array index
interface AABB { min: Vec2; max: Vec2 }
interface Interval { min: Metres; max: Metres } // vertical
type ConstraintSeverity = 'hard' | 'soft';
type ConstraintType =
  | 'outside_room'
  | 'door_swing'
  | 'fixture_access'
  | 'fixture_out_of_wall' // horizontal interval OR vertical extent
  | 'physical_collision'
  | 'clearance';
interface ValidationViolation {
  type: ConstraintType;
  severity: ConstraintSeverity;
  message: string;
  involvedObjectIds: string[];
  involvedWallIds?: string[];
  involvedFixtureIds?: string[];
}
interface ValidationResult {
  valid: boolean; // true only if no hard violations
  violations: ValidationViolation[];
}
interface SnapCandidate {

  targetType: 'grid' | 'wall' | 'wall_endpoint' | 'furniture_edge' | 'furniture_centre' | 'alignmen

  position: Vec2;
  distance: Metres;
  priority: number;
  reason: string;
}
type PolygonErrorCode =
  | 'SELF_INTERSECTION'
  | 'ZERO_AREA'
  | 'DEGENERATE_EDGE'
  | 'DUPLICATE_VERTEX'
  | 'TOO_FEW_VERTICES';
type ApplyErrorCode =
  | 'UNKNOWN_COMMAND_TYPE'
  | 'MISSING_ID'
  | 'ID_NOT_FOUND'
  | 'STRUCTURALLY_CORRUPT_COMMAND';
```

3. Domain Entities (Phase 1 set)

```
interface Project {
  schemaVersion: 1;
  id: string;
  units: 'metric' | 'imperial';
  rooms: Room[];
  furnitureDefinitions: FurnitureDefinition[];
}
interface Room {
  id: string;
  name: string;
  floorElevation: Metres; // default 0
  wallHeight: Metres;
  ceilingHeight: Metres;
  vertices: Vertex[]; // CCW, interior finished face, simple
  walls: WallSegment[];
  fixtures: Fixture[];
  furniture: FurnitureInstance[];
}
interface WallSegment {
  id: string;
  startVertexId: string; // stable vertex IDs, never array indices
  endVertexId: string;
  thickness: Metres; // default 0.15, expands outward
}
interface ClearancePolicy {
  side: 'front' | 'back' | 'left' | 'right' | 'all';
  offset: Metres;
  severity: 'hard' | 'soft';
}
interface FurnitureDefinition {
  id: string;
  name: string;
  category: string;
  defaultWidth: Metres;
  defaultLength: Metres;
  defaultHeight: Metres;
  clearancePolicies?: ClearancePolicy[];
}
interface FurnitureInstance {
  id: string;
  definitionId: string;
  roomId: string;
  position: Vec2; // centre
  elevation: Metres; // default 0
  rotation: number; // radians (quantized on commit)
  width: Metres;
  length: Metres;
  height: Metres;
  locked?: boolean;
  metadata?: {
```

```

vendor?: string;
sku?: string;
finishCode?: string;
unitCost?: number;
notes?: string;
};
}
interface Fixture {
  id: string;
  type: 'door' | 'window';
  wallId: string;
  offsetAlongWall: Metres; // along interior boundary
  width: Metres;
  height: Metres;
  elevation: Metres;
  // door-specific
  hingeSide?: 'left' | 'right';
  swingAngle?: number; // radians, default  $\pi/2$ 
  accessZoneDepth?: Metres; // additional depth beyond leaf
}

```

Clearance zone geometry (binding, mirrors V2 §3):

- Zone = rectangle extending **offset** outward from the corresponding footprint edge in the object's local (rotated) frame; **'all'** extends all four edges.
- **Room-boundary exemption:** zone overlap with the room boundary is not a violation. Zones are tested only against other furniture footprints, door swing sectors, and fixture access zones.
- **Clearance-vs-clearance overlap is not a violation.**

4. Command Schema

All commands are immutable. New IDs are supplied by the caller; **apply** never generates IDs. **Every command carries mandatory inverse data.** Undo = apply the inverse; redo = re-apply the original. Replay of any stored command must never throw.

```

type Command =
  | PlaceFurnitureCommand
  | MoveFurnitureCommand
  | ResizeFurnitureCommand
  | RotateFurnitureCommand
  | DeleteFurnitureCommand
  | PlaceFixtureCommand
  | EditWallCommand
  | CompositeCommand;
interface PlaceFurnitureCommand {
  type: 'PlaceFurniture';
  instance: FurnitureInstance; // full instance, pre-generated id, quantized
  // inverse: delete by instance.id
}
interface MoveFurnitureCommand {

```

```

    type: 'MoveFurniture';
    instanceId: string;
    from: Vec2; // REQUIRED (inverse data)
    to: Vec2; // quantized
  }
  interface ResizeFurnitureCommand {
    type: 'ResizeFurniture';
    instanceId: string;
    from: { position: Vec2; width: Metres; length: Metres }; // REQUIRED
    to: { position: Vec2; width: Metres; length: Metres }; // quantized; anchor logic
    lives in the interaction layer
  }
  interface RotateFurnitureCommand {
    type: 'RotateFurniture';
    instanceId: string;
    from: number; // REQUIRED (radians)
    to: number; // quantized radians
  }
  interface DeleteFurnitureCommand {
    type: 'DeleteFurniture';
    instanceId: string;
    snapshot: FurnitureInstance; // REQUIRED full pre-state (inverse)
  }
  interface PlaceFixtureCommand {
    type: 'PlaceFixture';
    fixture: Fixture; // full fixture, pre-generated id, quantized
    // inverse: delete by fixture.id
  }
  interface EditWallCommand {
    type: 'EditWall';
    roomId: string;
    from: { // REQUIRED full pre-state (inverse)
      vertices: Vertex[];
      walls: WallSegment[];
      // fixtures whose offsets must be restored if the edit invalidated them
      fixtureSnapshots?: Fixture[];
    };
    to: { // already validated vertices + walls
      vertices: Vertex[];
      walls: WallSegment[];
    };
  }
  interface CompositeCommand {
    type: 'Composite';
    commands: Command[]; // applied atomically left-to-right
    // inverse: CompositeCommand of the inverses in reverse order
  }

```

`apply(command: Command, state: Project): Project` is a pure, total function over well-formed commands.

5. Required Pure Modules

Module	Responsibility
<code>coordinates.ts</code>	<code>quantizeLinear(m)</code> , <code>quantizeRotation(rad)</code> , unit conversion, 3D axis mapping (World Y → Three Z, Elevation → Three Y)
<code>polygons.ts</code>	CCW normalize, validate (stable error codes), area, triangulation or equivalent containment prep
<code>footprints.ts</code>	Oriented rectangle (or polygon) footprint from centre + width + length + rotation; pure <code>resizeAboutAnchor(...)</code> helper (anchor math lives in the domain, not the interaction layer)
<code>collision.ts</code>	Convex intersection (SAT or equivalent) + vertical interval overlap
<code>containment.ts</code>	Footprint-in-room using boundary policy + EPSILON
<code>snapping.ts</code>	Generate candidates, rank by priority then distance, deterministic tie-break
<code>constraints.ts</code>	Door swing geometry, fixture-in-wall (horizontal + vertical), clearance evaluation (with room-boundary exemption) → structured ValidationResult
<code>validation.ts</code>	Orchestrate: containment → collision → fixture constraints → clearances
<code>commands.ts</code>	Command types + pure <code>apply</code>
<code>history.ts</code>	Undo/redo stack (§5c)
<code>serialize.ts</code>	Round-trip Project → JSON → Project (includes schemaVersion)

5a. `apply` Failure Semantics (binding)

- **Constraint rejection never reaches `apply`**. The commit pipeline (§6) rejects hard-constraint failures before a command is emitted.
- `apply` throws `ApplyError` with a stable `ApplyErrorCode` **only** for malformed commands: unknown type, missing/unknown IDs, structurally corrupt payloads.
- Replaying a stored (previously committed) command against any reachable state in the history timeline must never throw.
- **Composite atomicity (binding):** `apply(Composite)` folds sub-commands over immutable intermediate states. If any sub-command throws, the whole composite yields no state — `apply` never mutates input or intermediate state in place, so a failed composite can never leave a partially applied domain.
- Locked-object move/resize/rotate/delete are rejected **in the pipeline** (validation step), producing `ApplyErrorCode` never being reached; the pipeline surfaces the lock violation as a constraint message.

5b. Undo Mechanism (binding)

- Undo = **inverse-application** of the stored command (constructed from mandatory `from` / `snapshot` data). Redo = re-application of the original command.
- Validity is re-derived after every undo/redo step; no stored validity state exists.

5c. History Interface (minimum)

```
interface History {  
  push(command: Command): void; // only after successful commit  
  undo(state: Project): Project | null; // null when empty
```

```
redo(state: Project): Project | null; // null when at newest
canUndo(): boolean;
canRedo(): boolean;
// commands are stored in serialisable form for persistence
}
```

6. Canonical Commit Pipeline

```
Proposed transform (float)
↓
Resolve snap (optional)
↓
Quantize candidate values
- linear → 1 mm (Math.round semantics)
- rotation → nearest 0.1° (or active angular snap)
↓
Re-evaluate hard constraints on quantized values
↓
If valid → emit immutable command (IDs + inverse data present, values quantized) → new state
If invalid → reject, keep previous committed state
```

This pipeline is the single source of truth for every placement, move, resize, rotate, dimension, and fixture edit.

Already-invalid object escape rule (binding): an object whose *current committed state* already violates hard constraints (e.g. invalidated by a wall edit) is not deadlocked. The pipeline evaluates its proposed move/resize/rotate against the **delta** of violations: the operation commits if it introduces no new violation type/involved-object pair that the object did not already have, and does not increase any existing violation's severity. Otherwise rejected. This guarantees the user can always drag an invalid object toward validity (the violation set shrinks monotonically along the drag), and the B5 "Fix position" affordance remains the one-click exit.

Quantization assertion (binding): the "already quantized" check in `apply` (debug) is **tolerance-based** — values within half a quantization step + EPSILON pass. Never exact equality, so degree→radian conversion error ($\approx 1e-16$) and trig-based recomputation can never produce false assertion failures.

6a. EditWall Pipeline Exception (binding)

EditWall commands follow a **modified** pipeline: the quantized candidate polygon is validated for *polygon validity only* (CCW, simple, stable error codes). Furniture/fixture constraint outcomes are **not** blocking. The edit commits even if objects become invalid; after commit, a full re-validation pass runs and the UI derives invalid flags. (Mirrors V2 A3.)

Fixture invalidation during wall edits: fixtures whose `offsetAlongWall` no longer fits their wall segment (end-margin rule) are clamped by the interaction layer where possible; unclamped fixtures remain committed and are reported as `fixture_out_of_wall` (derived, non-blocking).

7. Door Swing Geometry Contract

- Hinge point = derived from wall segment start/end, `offsetAlongWall`, and `hingeSide`.
- Leaf is a rectangle of width = fixture.width, depth ≈ 0 (or a thin thickness).
- Swing sector = polygonal approximation of the arc from closed to `swingAngle` (default $\pi/2$).
- `accessZone` = additional rectangular or sector region of depth `accessZoneDepth` beyond the leaf.
- Intersection of any furniture footprint with the swing sector or access zone is a **hard** violation.
- Intersection of one door's swing/access zone with **another door's** swing/access zone is a **soft** violation (visual only).
- **Arc discretization (binding)**: swing and access-zone arcs are approximated with a maximum angular step of 5° or maximum inscribed error of 0.5 mm, whichever yields the finer polygon. The discretization constant is shared by the engine and all test suites, so no implementation or test can silently choose a coarser (false-negative) or finer (false-positive) approximation than the other.
- Non-90° angles are supported and must be tested.

8. Snap Priority & Tie-Break

Default priority values (higher = preferred):

Target	Priority
wall_endpoint	100
wall	80
furniture_edge	60
furniture_centre	50
alignment	40
grid	20

Tie-break when priority and distance are equal: stable order of targetType as listed above. Default `snapDistance` = 0.25 m. (Orientation snap at priority 70 is a V2/Appendix B concern, not Phase 1.)

9. Phase 1 Exit Criteria (testable)

1. Room polygon is CCW and valid, or rejected with a stable error code.
2. Furniture footprint is correctly generated for any rotation (including quantized angles).
3. Containment respects boundary policy + EPSILON.
4. After quantization, a corner that lands exactly on the boundary is classified as touching (valid).
5. Two objects collide only when both 2D footprints and vertical intervals overlap.
6. Door swing produces a hard violation when intersected (including non-90° angles); door-vs-door swing is soft.
7. Fixture whose interval falls outside its wall segment (end margin) **or** outside the wall's vertical extent is rejected as `fixture_out_of_wall`.
8. Snap candidates are generated, ranked by priority, and tie-broken deterministically.
9. Quantization + re-validation prevents "valid float $\rightarrow$ invalid after quantize" bugs for both translation and rotation.
10. `apply` is pure and total over well-formed commands; malformed commands throw stable `ApplyErrorCode`s; replaying any stored command never throws.

11. CompositeCommand applies atomically, appears as one history entry, and its inverse (reverse order) restores both objects.
12. Serialization round-trips within quantization rules (linear exact; rotation tolerant to $\leq 1e-9$ rad); `schemaVersion` is present.
13. Per-segment wall thickness is stored and does not alter the interior polygon.
14. Locked objects still collide; move/resize/rotate/delete on them are rejected in the pipeline.
15. Validity is derived; undo (inverse-application) restores geometry and therefore restores derived validity.
16. Clearance zones exempt the room boundary; clearance-vs-clearance overlap is not a violation.
17. `EditWall` commits under the pipeline exception and objects made invalid are reported by re-validation, not by blocking the edit.
18. Wall topology references vertices by stable ID; vertex insertion/deletion never invalidates other segments' references, and EditWall inverses restore geometry by ID.
19. An already-invalid object can be moved to any state that does not worsen its violation set (escape rule); the violation set shrinks monotonically toward validity.

10. Vitest Matrix (minimum)

Polygon — CCW, self-intersection → `SELF_INTERSECTION`, zero-area → `ZERO_AREA`, degenerate edge, duplicate vertex, negative coordinates, too few vertices.

Footprint — 0°, 45°, 90°, arbitrary angle, centre-pivot, after rotation quantization.

Containment — Fully inside, touching edge (EPSILON), crossing edge, fully outside, concave simple room.

Quantization edge — Corner quantized exactly onto boundary → touching (valid). Previously valid touching point never pushed $1e-6$ outside by quantization.

Collision — Overlapping, touching (EPSILON), separated, different elevations.

Vertical — Stacked, overlapping heights, touching heights.

Door swing — Clear, intersecting leaf, intersecting access zone, 90°, 180°, arbitrary angle, hinge-side variants, door-vs-door soft violation.

Fixture-in-wall — Fully inside segment, overlapping end margin, completely outside segment, vertical: window taller than wall, door elevated above 0.

Clearance — Zone overlapping furniture (hard/soft per policy), zone overlapping room boundary → no violation, zone-vs-zone → no violation, rotated object with side-specific policy.

Snap — Grid, wall, endpoint, priority ordering, equal-distance tie-break.

Quantize — 0.0004 → 0, 0.0005 → 0.001, 0.0006 → 0.001, negative values, rotation 45.0000001° → exact 45°.

Commands — Place → move → resize → rotate → undo (inverse) → redo. Composite (group move of two) → single history entry → undo restores both. Malformed command → stable `ApplyErrorCode`. Replay of stored command sequence → identical state.

EditWall — Valid vertices commit; self-intersecting polygon rejected; object made invalid by edit remains committed and reported by re-validation; inverse restores geometry and validity; vertex insertion/deletion preserves all other segments' vertex-ID references; malformed index-style reference → `ApplyErrorCode`.

Escape rule — Object invalidated by a wall edit: move that worsens violations → rejected; move that shrinks the violation set → commits; move into fully valid position → commits and clears the derived invalid flag.

Quantization assert — Command values within half-step + EPSILON of the quantization grid pass `apply`'s debug assert; exact-equality asserts are forbidden.

Locked — Move/resize/rotate/delete on locked instance rejected in pipeline; collision still occurs.

Serialize — Full project round-trip, schemaVersion present, linear exact, rotation tolerant ($\leq 1e-9$ rad).

Area — Polygon area returns correct m^2 for rectangle and concave simple polygon.

11. Out of Scope for Phase 1 (explicit)

- Any React / UI / store integration
- Konva or R3F
- Actual 2D/3D rendering
- Ghost placement interaction logic
- Live dimension overlay logic
- PDF export
- Multi-room navigation UI
- Full WALL / CEILING anchor system
- Surface-snap elevation (A8) — intentionally deferred
- Assemblies UI
- Rooms with holes (inner rings)
- Window-specific constraints beyond generic fixture-in-wall validation (none additional in Phase 1)
- Orientation snap (Appendix B B2) — interaction-layer concern
- Broad-phase spatial index: not *required*, but an optional AABB prefilter (~30 lines, filters pairs by bounding-box overlap before SAT) **may** be implemented in Phase 1 behind the unchanged public `validation.ts` API. Brute force remains the accepted default for ≤ 100 objects.

12. Implementation Discretion

The developer may choose:

- Exact convex intersection algorithm (SAT or other)
- Exact concave containment method (triangulation, winding number, library, etc.)
- Internal data structures and module layout
- Any pure helper libraries

provided the invariants, public contracts, command purity, `apply` totality + error codes, and test matrix remain satisfied.

End of Phase 1 — Domain & Geometry Implementation Contract (Rev 2)